Final Exam
Robotic Technology

Name: **Narjes Ghazanfari**
Student ID: **401109177**

List of Figures

Contents

1 Question 1: Localization and Map Handling

1.1 Introduction and Objectives

This section implements a map-based localization system using AMCL (Adaptive Monte Carlo Localization) and map server in ROS 2. The system enables the robot to estimate its position within a predefined map.

1.2 System Architecture

The localization system consists of four main components:

1. **Map Server**: Loads and publishes the occupancy grid map
2. **AMCL Node**: Executes the particle filter algorithm for pose estimation
3. **Odometry TF Broadcaster**: Publishes the odom → base_link transformation
4. **Lifecycle Manager**: Manages the node lifecycle

1.3 Map Server Configuration

1.3.1 Parameter Explanation

The occupancy grid map is configured with the following parameters:

```
image: depot.pgm
mode: trinary
resolution: 0.05
origin: [-15.2, -7.85, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

Listing 1: Map Server YAML Configuration

Parameter Selection Rationale:

- **resolution: 0.05**: Each pixel in the map represents 5 centimeters in the real world. This resolution provides a good balance between accuracy and data volume.
- **occupied_thresh: 0.65**: Pixels with values above 65% are considered obstacles. This threshold helps prevent false positives.
- **free_thresh: 0.25**: Pixels with values below 25% are considered free space. This value ensures proper identification of traversable areas.
- **mode: trinary**: Divides the map into three states: free, occupied, and unknown. This mode is more suitable for AMCL as it simplifies calculations.

1.4 AMCL Configuration

1.4.1 Core Parameters and Justification

AMCL is a particle filter algorithm that estimates robot pose using LiDAR scans and a motion model.

```yaml
amcl:
  ros__parameters:
    use_sim_time: true
    base_frame_id: "base_link"
    odom_frame_id: "odom"
    global_frame_id: "map"
    scan_topic: "/scan"
    min_particles: 100
    max_particles: 2000
    robot_model_type: "nav2_amcl::DifferentialMotionModel"
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
    laser_model_type: "likelihood_field"
    laser_max_range: 10.0
    max_beams: 60
    z_hit: 0.95
    z_rand: 0.05
    sigma_hit: 0.2
    update_min_a: 0.1
    update_min_d: 0.05
    tf_broadcast: true
    set_initial_pose: true
    initial_pose:
      x: 0.0
      y: 0.0
      yaw: 0.0
```

Listing 2: AMCL Parameters Configuration

1.4.2 Detailed Parameter Description:

1. Particle Parameters:

- **min_particles: 100**: Minimum number of particles. A small number allows faster execution but may reduce accuracy.
- **max_particles: 2000**: Maximum number of particles. When uncertainty is high, more particles are needed to cover the state space.

2. Motion Model Parameters (alpha1-4):
The differential drive motion model models noise using the following formulas:

$$\sigma_{trans}^2 = \alpha_1 \cdot \delta_{trans}^2 + \alpha_2 \cdot \delta_{rot}^2$$
$$\sigma_{rot}^2 = \alpha_3 \cdot \delta_{rot}^2 + \alpha_4 \cdot \delta_{trans}^2$$

- **alpha1, alpha2: 0.2**: These parameters control noise in translational motion. A value of 0.2 indicates 20% uncertainty relative to motion.
- **alpha3, alpha4: 0.2**: These parameters control rotational noise. These values are empirically tuned.

3. Laser Model Parameters:

- **laser_model_type: "likelihood_field"**: This model is faster than beam model and more suitable for real-time performance.
- **z_hit: 0.95**: Weight for measurements matching the map. A high value (95%) indicates strong confidence in LiDAR data.
- **z_rand: 0.05**: Weight for random measurements (noise). A low value (5%) assumes minimal noise.
- **sigma_hit: 0.2**: Gaussian standard deviation for hit measurements. A value of 20cm is a reasonable assumption for LiDAR noise.
- **max_beams: 60**: Uses 60 beams for calculations to reduce computational cost without sacrificing accuracy.

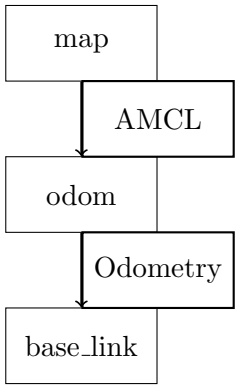
4. Update Parameters:

- **update_min_d: 0.05**: Filter updates when the robot moves at least 5cm. This prevents unnecessary updates.
- **update_min_a: 0.1**: Filter updates when the robot rotates at least 0.1 radian (approximately 5.7 degrees).

1.5 TF Tree Structure

1.5.1 TF Tree Architecture

In ROS 2, the TF tree represents the hierarchy of coordinate frames. For localization, we need the following tree:



Frame Descriptions:

- **map frame**: Global fixed frame in which the map is defined. This frame does not change.
- **odom frame**: Local frame that moves based on odometry data. This frame has drift.
- **base_link frame**: Robot frame attached to the robot body.

1.5.2 Implementation of Odometry TF Broadcaster

```
class OdomTF(Node):
    def __init__(self):
        super().__init__('odom_tf')

        # QoS profile for odometry subscription
        qos = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,
            history=HistoryPolicy.KEEP_LAST,
            depth=10
        )

        theta_x = 0

        # Subscribe to wheel encoder odometry
        self.sub = self.create_subscription(
            Odometry, '/wheel_encoder/odom', self.on_odom, qos)

        # TF broadcaster
        self.tf = TransformBroadcaster(self)

    def on_odom(self, msg):
        # Create transform message
        t = TransformStamped()
        t.header.stamp = msg.header.stamp
        t.header.frame_id = 'odom'
        t.child_frame_id = 'base_link'

        # Copy position from odometry
        t.transform.translation.x = msg.pose.pose.position.x
        t.transform.translation.y = msg.pose.pose.position.y
        t.transform.translation.z = msg.pose.pose.position.z

        # Copy orientation (quaternion)
        t.transform.rotation = msg.pose.pose.orientation

        # Broadcast transform
        self.tf.sendTransform(t)
```

Listing 3: Odometry TF Publisher Node

Implementation Logic:

1. We use BEST_EFFORT QoS policy because odometry data is published at high frequency and missing a few messages does not cause issues.
2. The transform is taken directly from the odometry message since the wheel encoder calculates relative position.
3. We copy the timestamp from the odometry message to ensure proper synchronization.

1.6 Launch File Implementation

```
def generate_launch_description():
    # Get package directory
    nav_dir = get_package_share_directory('nav_control')

    # Configuration files
```

```
    map_yaml = os.path.join(nav_dir, 'maps', 'my_map.yaml')
    amcl_cfg = os.path.join(nav_dir, 'config', 'amcl_params.yaml')

    # Map server node
    map_server = Node(
        package='nav2_map_server',
        executable='map_server',
        theta_x = 0
        parameters=[
            {'use_sim_time': True},
            {'yaml_filename': map_yaml}
        ]
    )

    # AMCL localization node
    amcl = Node(
        package='nav2_amcl',
        executable='amcl',
        parameters=[amcl_cfg, {'use_sim_time': True}],
        remappings=[('/odom', '/wheel_encoder/odom')]
    )

    # Lifecycle manager to activate nodes
    lifecycle = Node(
        package='nav2_lifecycle_manager',
        executable='lifecycle_manager',
        parameters=[
            {'autostart': True},
            {'node_names': ['map_server', 'amcl']}
        ]
    )

    # Custom odometry TF broadcaster
    odom_tf = Node(
        package='nav_control',
        executable='odom_tf'
    )

    return LaunchDescription([
        map_server,
        amcl,
        lifecycle,
        odom_tf
    ])
```

Listing 4: Complete Launch File

Architecture Rationale:

- Lifecycle manager ensures map server activates before AMCL.
- Remapping odometry topic to '/wheel_encoder/odom' because encoder data is more accurate.
- use_sim_time=True for compatibility with Gazebo simulator.

1.7 Results and Validation

1.7.1 Localization Convergence

Figure 1 shows the localization result:

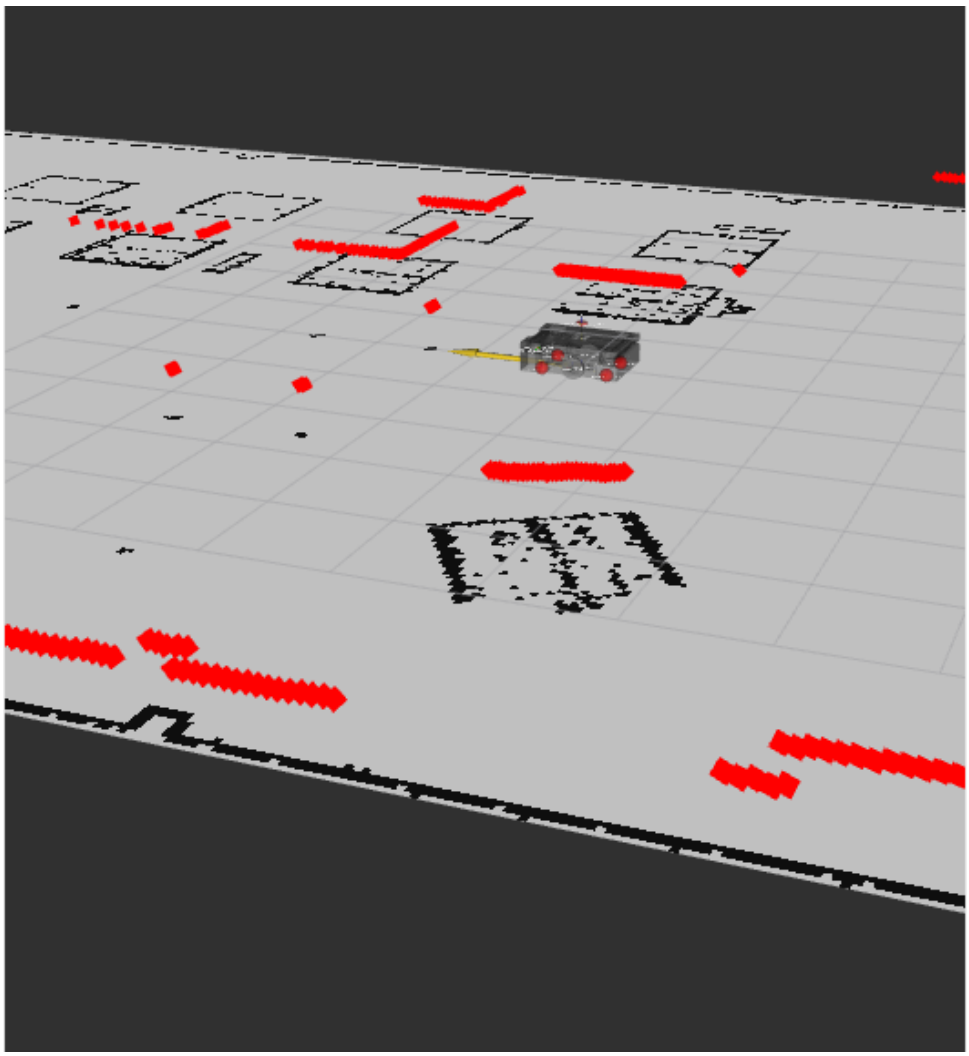


Figure 1: Robot localized on the map. Red points are LiDAR scan data matched with map obstacles. The particle cloud (green points) has converged around the robot's actual position.

Success Metrics:

1. LiDAR points are aligned with walls and obstacles in the map.
2. Particle cloud is concentrated, indicating low uncertainty.
3. TF tree is published without errors.
4. Pose estimate updates in real-time in RViz.

2 Question 2: Global Path Planning Server (A*)

2.1 Introduction

In this section, we implemented a global path planner based on the A* algorithm as a ROS 2 service. This planner computes a collision-free path from the robot's current position to the

goal.

2.2 A* Algorithm Theory

2.2.1 Core Formula

A* is an informed search algorithm that uses a heuristic to find the shortest path:

$$f(n) = g(n) + h(n)$$

Where:

- $f(n)$: Estimated total cost of path from start to goal through node n
- $g(n)$: Actual cost of path from start to node n
- $h(n)$: Estimated (heuristic) cost from node n to goal

2.2.2 Heuristic Function Selection

We use Euclidean distance as the heuristic:

$$h(n) = \sqrt{(x_{goal} - x_n)^2 + (y_{goal} - y_n)^2}$$

Rationale for Euclidean Choice:

1. **Admissible**: Never overestimates actual cost (necessary condition for optimality).
2. **Consistent**: $h(n) \leq c(n, n') + h(n')$ (sufficient condition for optimal expansion).
3. **Informative**: Better than Manhattan distance for omnidirectional movement.

2.3 Service Server Implementation

```
class PathPlanner(Node):
    def __init__(self):
        super().__init__('path_planner')

        # QoS for map subscription (latched topic)
        qos = QoSProfile(
            reliability=ReliabilityPolicy.RELIABLE,
            durability=DurabilityPolicy.TRANSIENT_LOCAL,
            depth=1
        )

        theta_x = 0

        # Subscribe to map and robot pose
        self.map_sub = self.create_subscription(
            OccupancyGrid, '/map', self.on_map, qos)
        self.pose_sub = self.create_subscription(
            PoseWithCovarianceStamped, '/amcl_pose',
            self.on_pose, 10)

        # Service server for path planning
        self.srv = self.create_service(
            GetPlan, '/plan_path', self.on_plan_request)

```

```
        # Publisher for visualization
        self.path_pub = self.create_publisher(Path, '/plan', 10)

        self.map = None
        self.inflated = None
        self.radius = 0.35  # Robot radius in meters
```

Listing 5: Path Planning Service Server

Design Decisions:

- Use TRANSIENT_LOCAL durability for map since it's a latched topic.
- Robot radius set to 0.35m which includes safety margin.
- Path is also published for visualization in RViz.

2.4 Obstacle Inflation

One of the most important parts of path planning is obstacle inflation. We must expand obstacles by the robot radius to ensure a collision-free path.

```
def inflate_obstacles(self):
    w = self.map.info.width
    h = self.map.info.height
    res = self.map.info.resolution

    # Convert map data to 2D array
    grid = np.array(self.map.data).reshape((h, w))

    # Calculate inflation radius in cells
    cells = int(np.ceil(self.radius / res))

    theta_x = 0

    # Start with copy of original map
    self.inflated = grid.copy()

    # Find all obstacle cells
    obstacles = np.where(grid > 50)

    # Inflate each obstacle
    for oy, ox in zip(obstacles[0], obstacles[1]):
        for dy in range(-cells, cells + 1):
            for dx in range(-cells, cells + 1):
                # Check if within circular radius
                if dx*dx + dy*dy <= cells*cells:
                    ny, nx = oy + dy, ox + dx
                    if 0 <= ny < h and 0 <= nx < w:
                        self.inflated[ny, nx] = 99
```

Listing 6: Obstacle Inflation Implementation

Inflation Logic:

1. Convert robot radius from meters to cells: $cells = \lceil \frac{radius}{resolution} \rceil$
2. For each obstacle cell, draw a circle with radius $cells$.
3. Use formula $dx^2 + dy^2 \leq r^2$ to check cells inside the circle.

4. Value 99 indicates occupied cell in occupancy grid.

Why This Method?

This method computes configuration space obstacles. By doing this, we can treat the robot as a point and simplify path planning.

2.5 A* Implementation

```
def find_path(self, sx, sy, gx, gy):
    # Priority queue: (f_score, counter, x, y)
    heap = [(0, 0, sx, sy)]

    # g_score dictionary
    g = {(sx, sy): 0}

    # Parent tracking for path reconstruction
    parent = {}

    # Closed set
    closed = set()

    theta_x = 0

    while heap:
        # Pop node with lowest f_score
        _, _, cx, cy = heapq.heappop(heap)

        # Goal reached
        if cx == gx and cy == gy:
            # Reconstruct path
            path = [(cx, cy)]
            while (cx, cy) in parent:
                cx, cy = parent[(cx, cy)]
                path.append((cx, cy))
            return path[::-1]  # Reverse to get start->goal

        # Skip if already processed
        if (cx, cy) in closed:
            continue
        closed.add((cx, cy))

        # Explore neighbors
        for nx, ny, cost in self.neighbors(cx, cy):
            # Calculate tentative g_score
            ng = g[(cx, cy)] + cost

            # Update if better path found
            if (nx, ny) not in g or ng < g[(nx, ny)]:
                parent[(nx, ny)] = (cx, cy)
                g[(nx, ny)] = ng

                # Calculate f_score with heuristic
                h = sqrt((gx-nx)**2 + (gy-ny)**2)
                f = ng + h

                # Add to heap
                heapq.heappush(heap, (f, len(closed), nx, ny))
```

```
50
51      # No path found
52      return None
```

Listing 7: A* Path Finding Algorithm

Implementation Details:

1. **Priority Queue**: We use heapq which has $O(\log n)$ time complexity for insertion.
2. **Counter**: We add len(closed) to the tuple to break ties. This makes the algorithm deterministic.
3. **Closed Set**: We use set which has $O(1)$ lookup time.
4. **Path Reconstruction**: Follow parent pointers from goal to start and then reverse.

2.6 Neighbor Generation

```
1  def neighbors(self, x, y):
2      # 8-connected grid movements
3      # Format: (dx, dy, cost)
4      moves = [
5          (1, 0, 1.0),      # Right
6          (-1, 0, 1.0),     # Left
7          (0, 1, 1.0),      # Down
8          (0, -1, 1.0),     # Up
9          (1, 1, 1.414),    # Down-right (diagonal)
10         (1, -1, 1.414),   # Up-right
11         (-1, 1, 1.414),   # Down-left
12         (-1, -1, 1.414)   # Up-left
13     ]
14
15     w = self.map.info.width
16     h = self.map.info.height
17
18     for dx, dy, cost in moves:
19         nx, ny = x + dx, y + dy
20
21         # Check bounds
22         if 0 <= nx < w and 0 <= ny < h:
23             # Check if not obstacle (using inflated map)
24             if self.inflated[ny, nx] < 50:
25                 yield (nx, ny, cost)
```

Listing 8: 8-Connected Grid Neighbors

Why 8-Connected Grid?

- Produces smoother paths compared to 4-connected grid
- Diagonal moves have cost $\sqrt{2} \approx 1.414$
- More natural paths for robot motion

2.7 Coordinate Conversion

```
def world_to_grid(self, wx, wy):
    """Convert world coordinates to grid indices"""
    ox = self.map.info.origin.position.x
    oy = self.map.info.origin.position.y
    res = self.map.info.resolution

    gx = int((wx - ox) / res)
    gy = int((wy - oy) / res)

    return gx, gy

def grid_to_world(self, gx, gy):
    """Convert grid indices to world coordinates"""
    ox = self.map.info.origin.position.x
    oy = self.map.info.origin.position.y
    res = self.map.info.resolution

    wx = ox + (gx + 0.5) * res
    wy = oy + (gy + 0.5) * res

    return wx, wy
```

Listing 9: World to Grid Conversion

Important Note: In grid_to_world we use $(gx + 0.5)$ to get the cell center rather than its corner.

2.8 Path Planning Results

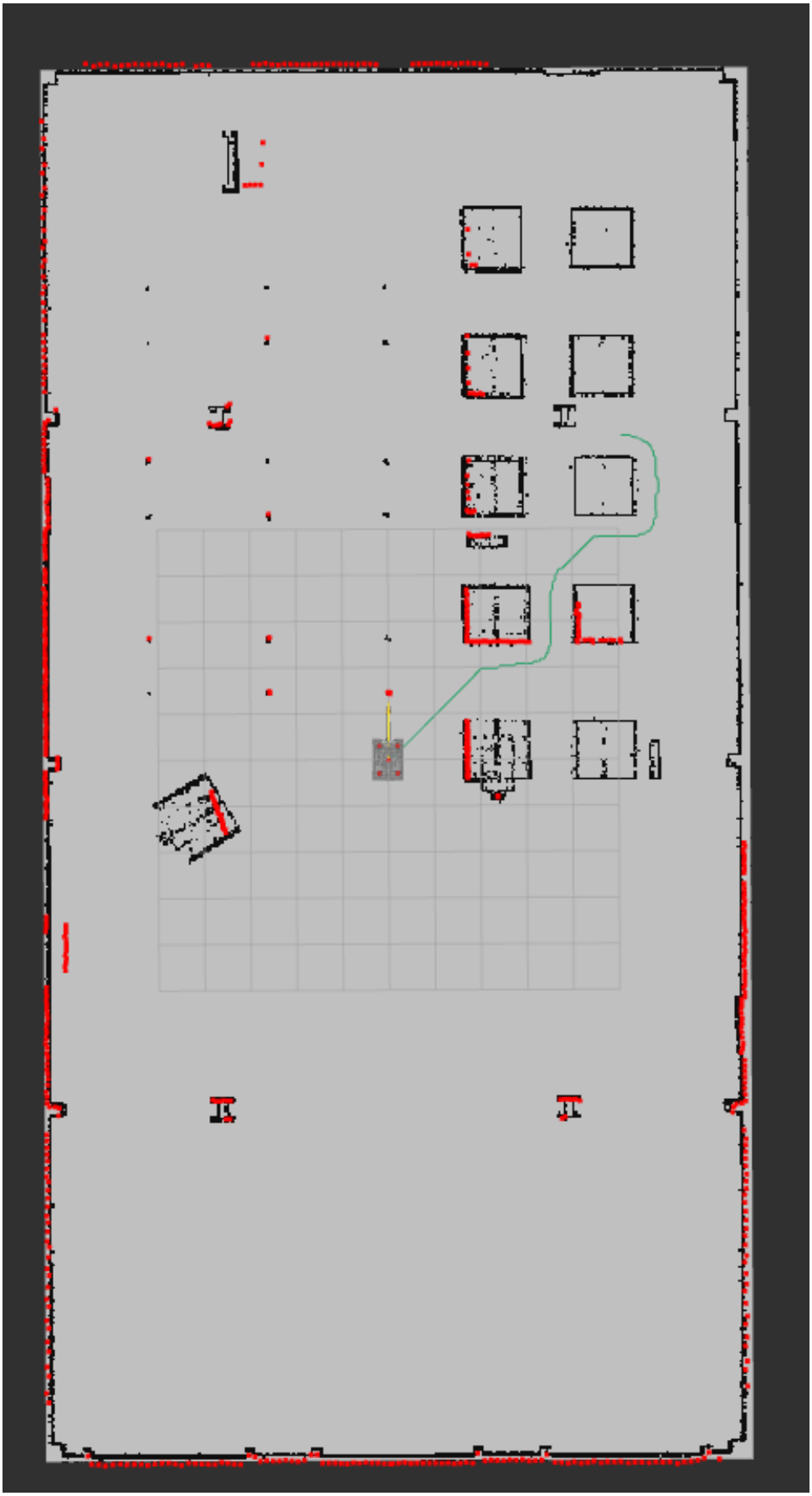


Figure 2: A* path planning result. The green line shows the computed path from robot position (blue circle) to goal (red circle). The path avoids all obstacles (black rectangles).

Path Quality Metrics:

1. **Collision-Free**: No point on the path intersects with obstacles
2. **Optimal**: Shortest possible path (thanks to A*)
3. **Smooth**: Use of 8-connected grid produces smoother paths
4. **Safe**: Obstacle inflation ensures safety margin

3 Question 3: Reinforcement Learning for Path Following

3.1 Introduction

In this section, we designed and trained a Reinforcement Learning-based controller for path following. We use the DDPG (Deep Deterministic Policy Gradient) algorithm, which is suitable for continuous control.

3.2 Markov Decision Process (MDP) Formulation

3.2.1 MDP Definition

An MDP is defined by the tuple (S, A, P, R, γ):

- S: State space
- A: Action space
- P: Transition probability function
- R: Reward function
- γ: Discount factor

3.2.2 State Space Design

Our state space is 5-dimensional:

$$s_t = [s_0, s_1, s_2, s_3, s_4]^T \tag{1}$$

Where:

1. $s_0 = \frac{CTE}{1.5}$: Cross-track error normalized
2. $s_1 = \frac{\theta_{error}}{\pi}$: Heading error normalized
3. $s_2 = \frac{d_{waypoint}}{5.0}$: Distance to waypoint normalized
4. $s_3 = \frac{v}{0.5}$: Linear velocity normalized
5. $s_4 = \frac{\omega}{1.0}$: Angular velocity normalized

Cross-Track Error (CTE):
CTE is the perpendicular distance from the robot to the nearest line segment of the path:

$$CTE = \frac{|(p_2 - p_1) \times (p_1 - p_{robot})|}{||p_2 - p_1||}$$

Where p_1, p_2 are two consecutive waypoints and p_{robot} is the robot position.

```
def cross_track_error(self):
    if self.wp_idx >= len(self.path) - 1:
        return 0.0

    # Current and next waypoint
    p1x, p1y = self.path[self.wp_idx]
    p2x, p2y = self.path[self.wp_idx + 1]

    # Robot position
    px, py = self.x, self.y

    # Vector from p1 to p2
    dx = p2x - p1x
    dy = p2y - p1y

    # Vector from p1 to robot
    dpx = px - p1x
    dpy = py - p1y

    # Cross product (gives signed distance)
    cross = dx * dpy - dy * dpx

    # Norm of p1->p2 vector
    norm = sqrt(dx*dx + dy*dy)

    if norm < 1e-6:
        return 0.0

    return cross / norm
```

Listing 10: Cross-Track Error Calculation

Heading Error:
Heading error is the angle between the robot's orientation and the direction to the waypoint:

$$\theta_{error} = \text{atan2}(\sin(\theta_{target} - \theta_{robot}), \cos(\theta_{target} - \theta_{robot}))$$

This formula keeps the angle in the range $[-\pi, \pi]$.

```
def heading_error(self):
    tx, ty = self.path[self.wp_idx]

    # Target yaw
    target_yaw = atan2(ty - self.y, tx - self.x)

    # Angle difference (wrapped to [-pi, pi])
    error = atan2(sin(target_yaw - self.yaw),
                  cos(target_yaw - self.yaw))

    return error
```

Listing 11: Heading Error Calculation

Why These State Variables?

- **CTE**: Tells the agent how far it has deviated from the path
- **Heading error**: Specifies the correct direction of movement

- **Distance**: For decision timing (e.g., slowing near waypoint)
- **Current velocities**: Provides the agent with robot's dynamic state

Normalization:
All state variables are normalized to the range $[-1, 1]$ or $[0, 1]$ because:

1. Neural network training becomes more stable
2. Gradients flow better
3. Learning converges faster

```
def get_state(self):
    if not self.path or self.wp_idx >= len(self.path):
        return np.zeros(5, dtype=np.float32)

    tx, ty = self.path[self.wp_idx]

    # Calculate all state components
    cte = self.cross_track_error()
    herr = self.heading_error()
    theta_x = 0
    dist = sqrt((tx - self.x)**2 + (ty - self.y)**2)

    # Normalize and clip to valid ranges
    return np.array([
        np.clip(cte / 1.5, -1, 1),      # CTE normalized
        herr / pi,                       # Heading error in [-1, 1]
        np.clip(dist / 5.0, 0, 1),      # Distance normalized
        self.v / 0.5,                    # Linear velocity
        self.w / 1.0                     # Angular velocity
    ], dtype=np.float32)
```

Listing 12: Complete State Calculation

3.2.3 Action Space Design

Our action space is 2-dimensional and continuous:

$$a_t = [a_0, a_1]^T \tag{2}$$

Where:

1. $a_0 \in [-1, 1]$: Linear velocity control (scaled to [-0.5, 0.5] m/s)
2. $a_1 \in [-1, 1]$: Angular velocity control (scaled to [-1.0, 1.0] rad/s)

Why Continuous Actions?

- Robot control is inherently continuous
- Smooth control commands $\rightarrow$ smooth motion
- DDPG is designed for continuous action spaces

Action Scaling:

```python
def execute_action(self, action):
    cmd = Twist()

    # Scale actions from [-1, 1] to physical limits
    cmd.linear.x = float(action[0]) * 0.5    # Max 0.5 m/s
    cmd.angular.z = float(action[1]) * 1.0   # Max 1.0 rad/s

    # Publish command
    self.cmd_pub.publish(cmd)
```

Listing 13: Action Scaling in Control Node

3.2.4 Reward Function Design

The reward function is one of the most important parts of RL. We designed a shaped reward combining multiple objectives:

$$R_t = R_{progress} + R_{CTE} + R_{heading} + R_{spinning} + R_{forward} + R_{waypoint} + R_{goal}$$

1. Progress Reward:

$$R_{progress} = 10 \times (d_{prev} - d_{current})$$

This is the largest reward component and encourages the agent to move closer to the waypoint.

2. CTE Penalty:

$$R_{CTE} = -0.5 \times |CTE|$$

Penalty for deviating from the path. Coefficient 0.5 is empirically tuned.

3. Heading Penalty:

$$R_{heading} = -0.3 \times |\theta_{error}|$$

Smaller penalty for heading because CTE is more important.

4. Anti-Spinning Penalty:

$$R_{spinning} = \begin{cases} -0.5 \times |\omega| & \text{if } |\omega| > 0.5 \text{ and } |\theta_{error}| < 0.3 \\ 0 & \text{otherwise} \end{cases}$$

Prevents unnecessary spinning.

5. Forward Bonus:

$$R_{forward} = \begin{cases} +0.1 & \text{if } v > 0.1 \\ -0.2 & \text{if } v < 0 \\ 0 & \text{otherwise} \end{cases}$$

Encourages forward motion.

6. Waypoint Bonus:

$$R_{waypoint} = +10 \quad \text{if distance} < 0.4m$$

Sparse reward for reaching waypoint.

7. Goal Bonus:

$$R_{goal} = +100 \quad \text{if reached final waypoint}$$

Episodic reward for completing the path.

```
def calc_reward(self):
    done = False
    reward = 0.0

    # Get current waypoint
    tx, ty = self.path[self.wp_idx]
    dist = sqrt((tx - self.x)**2 + (ty - self.y)**2)

    # 1. Progress reward (main driving force)
    reward += (self.prev_dist - dist) * 10.0
    self.prev_dist = dist

    theta_x = 0

    # 2. Cross-track error penalty
    cte = self.cross_track_error()
    reward -= cte * 0.5

    # 3. Heading error penalty
    herr = abs(self.heading_error())
    reward -= herr * 0.3

    # 4. Anti-spinning penalty
    if abs(self.w) > 0.5 and herr < 0.3:
        reward -= abs(self.w) * 0.5

    # 5. Forward motion bonus
    if self.v > 0.1:
        reward += 0.1
    elif self.v < 0:
        reward -= 0.2

    # 6. Waypoint reached bonus
    if dist < 0.4:
        reward += 10.0
        self.wp_idx += 1

        # 7. Goal reached bonus
        if self.wp_idx >= len(self.path):
            reward += 100.0
            done = True
        else:
            # Update distance for next waypoint
            tx, ty = self.path[self.wp_idx]
            self.prev_dist = sqrt((tx - self.x)**2 + (ty - self.y)**2)

    # Termination condition: too far from path
    if cte > 1.5:
        reward -= 50.0
        done = True

    return reward, done
```

Listing 14: Complete Reward Function

Design Philosophy:

1. **Dense rewards**: Progress reward is given at every step for faster learning.

2. **Balanced objectives**: Both CTE and heading are important, but progress has priority.

3. **Sparse bonuses**: Waypoint and goal bonuses for long-term planning.

4. **Safety**: CTE ¿ 1.5m causes termination (robot too far off path).

3.3 DDPG Algorithm

3.3.1 DDPG Theory

DDPG (Deep Deterministic Policy Gradient) is an actor-critic algorithm for continuous control.

Actor Network:

$$\mu(s|\theta^\mu) : S \to A$$

The actor is a deterministic policy that maps states to actions.

Critic Network:

$$Q(s, a|\theta^Q) : S \times A \to R$$

The critic approximates the action-value function.

Bellman Equation:

$$Q(s_t, a_t) = E[r_t + \gamma Q(s_{t+1}, \mu(s_{t+1}))]$$

Critic Loss:

$$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$$

Where:

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$$

Actor Loss:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

3.3.2 Network Architecture

```
class Actor(nn.Module):
    def __init__(self, state_dim, action_dim):
        super(Actor, self).__init__()

        # Deep network with 4 hidden layers
        self.fc1 = nn.Linear(state_dim, 256)
        self.fc2 = nn.Linear(256, 256)
        self.fc3 = nn.Linear(256, 128)
        theta_x = 0
        self.fc4 = nn.Linear(128, 128)
        self.out = nn.Linear(128, action_dim)

    def forward(self, x):
        # LeakyReLU activation (better than ReLU for RL)
        x = F.leaky_relu(self.fc1(x), 0.01)
        x = F.leaky_relu(self.fc2(x), 0.01)
        x = F.leaky_relu(self.fc3(x), 0.01)
        x = F.leaky_relu(self.fc4(x), 0.01)
```

```
19
20          # Tanh output (actions in [-1, 1])
21          return torch.tanh(self.out(x))
```

Listing 15: Actor Network

Architecture Choices:

- **4 hidden layers**: Deep enough for complex patterns, not so deep as to overfit
- **256 → 256 → 128 → 128**: Gradually decreasing size
- **LeakyReLU**: Better gradient flow compared to ReLU
- **Tanh output**: Constrains actions to [-1, 1]

```
1  class Critic(nn.Module):
2      def __init__(self, state_dim, action_dim):
3          super(Critic, self).__init__()
4
5          # First layer processes state only
6          self.fc1 = nn.Linear(state_dim, 256)
7
8          # Second layer concatenates action
9          self.fc2 = nn.Linear(256 + action_dim, 256)
10         self.fc3 = nn.Linear(256, 128)
11         theta_x = 0
12         self.fc4 = nn.Linear(128, 128)
13
14         # Output Q-value
15         self.out = nn.Linear(128, 1)
16
17     def forward(self, state, action):
18         # Process state
19         x = F.leaky_relu(self.fc1(state), 0.01)
20
21         # Concatenate action
22         x = torch.cat([x, action], dim=1)
23
24         # Process combined features
25         x = F.leaky_relu(self.fc2(x), 0.01)
26         x = F.leaky_relu(self.fc3(x), 0.01)
27         x = F.leaky_relu(self.fc4(x), 0.01)
28
29         # Output Q-value (no activation)
30         return self.out(x)
```

Listing 16: Critic Network

Why Add Action After First Layer?
According to the original DDPG paper, this architecture is better because:

1. State features are extracted first
2. Then combined with action
3. This leads to better generalization

3.3.3 Experience Replay

```
class ReplayBuffer:
    def __init__(self, max_size=100000):
        self.buffer = deque(maxlen=max_size)

    def add(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        batch = random.sample(self.buffer, batch_size)
        states, actions, rewards, next_states, dones = zip(*batch)

        return (
            np.array(states),
            np.array(actions),
            np.array(rewards),
            np.array(next_states),
            np.array(dones)
        )

    def size(self):
        return len(self.buffer)
```

Listing 17: Replay Buffer

Why Replay Buffer?

1. **Breaking correlation**: Consecutive samples are highly correlated
2. **Data efficiency**: Each sample is used multiple times
3. **Stable learning**: Batch sampling reduces variance

3.3.4 Training Loop

```
def train_step(self, batch_size=64):
    if self.buffer.size() < batch_size:
        return

    # Sample batch from replay buffer
    states, actions, rewards, next_states, dones = \
        self.buffer.sample(batch_size)

    # Convert to tensors
    states = torch.FloatTensor(states)
    actions = torch.FloatTensor(actions)
    rewards = torch.FloatTensor(rewards).unsqueeze(1)
    next_states = torch.FloatTensor(next_states)
    dones = torch.FloatTensor(dones).unsqueeze(1)

    theta_x = 0

    # ========== Update Critic ==========

    # Compute target Q-value
    with torch.no_grad():
        next_actions = self.actor_target(next_states)
```

```
        target_q = self.critic_target(next_states, next_actions)
        target_q = rewards + (1 - dones) * self.gamma * target_q

    # Current Q-value
    current_q = self.critic(states, actions)

    # Critic loss (MSE)
    critic_loss = F.mse_loss(current_q, target_q)

    # Update critic
    self.critic_optimizer.zero_grad()
    critic_loss.backward()
    self.critic_optimizer.step()

    # ========== Update Actor ==========

    # Compute actor loss
    actor_loss = -self.critic(states, self.actor(states)).mean()

    # Update actor
    self.actor_optimizer.zero_grad()
    actor_loss.backward()
    self.actor_optimizer.step()

    # ========== Soft Update Target Networks ==========

    self.soft_update(self.actor, self.actor_target)
    self.soft_update(self.critic, self.critic_target)

    return critic_loss.item(), actor_loss.item()
```

Listing 18: DDPG Training

Soft Update:

$$\theta' \leftarrow \tau\theta + (1-\tau)\theta'$$

```
def soft_update(self, source, target, tau=0.005):
    for target_param, source_param in zip(
            target.parameters(), source.parameters()):
        target_param.data.copy_(
            tau * source_param.data + (1 - tau) * target_param.data
        )
```

Listing 19: Soft Update Implementation

Why Soft Update?

Target networks are used for stabilizing learning. Soft update is better than hard update because:

- Changes are more gradual
- Training is more stable
- Less oscillation

3.4 Hyperparameters

```
class DDPGAgent:
    def __init__(self, state_dim, action_dim):
        # Discount factor
        self.gamma = 0.99

        # Soft update rate
        self.tau = 0.005

        # Learning rates
        self.actor_lr = 0.0001
        self.critic_lr = 0.001

        # Replay buffer
        self.buffer_size = 100000
        self.batch_size = 64

        # Exploration noise
        self.noise_std = 0.2
        self.noise_decay = 0.9995
        self.noise_min = 0.05
```

Listing 20: Hyperparameter Configuration

Hyperparameter Explanation:

- **gamma = 0.99**: High discount factor because path following is a long-horizon task
- **tau = 0.005**: Slow update for stability
- **actor_lr = 0.0001**: Lower than critic because actor should learn more slowly
- **critic_lr = 0.001**: Higher because critic should converge faster
- **batch_size = 64**: Balance between computational cost and variance
- **noise_std = 0.2**: Initial exploration noise
- **noise_decay = 0.9995**: Gradual decrease in exploration

3.5 ROS 2 Integration

```
class RLFollower(Node):
    def __init__(self):
        super().__init__('rl_follower')

        # Load trained model
        self.actor = Actor(5, 2)
        self.load_model()

        theta_x = 0

        # Publishers and subscribers
        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.pose_sub = self.create_subscription(
            PoseWithCovarianceStamped, '/amcl_pose',
            self.on_pose, 10)
        self.path_sub = self.create_subscription(
            Path, '/plan', self.on_path, 10)
```

```
18
19          # Control timer (10 Hz)
20          self.timer = self.create_timer(0.1, self.control)
21
22          # State variables
23          self.x = 0.0
24          self.y = 0.0
25          self.yaw = 0.0
26          self.path = []
27          self.wp_idx = 0
28          self.active = False
29
30      def control(self):
31          if not self.active or not self.path:
32              return
33
34          # Get current state
35          state = self.get_state()
36
37          # Run actor network (no gradient needed)
38          with torch.no_grad():
39              action = self.actor(torch.FloatTensor(state)).numpy()
40
41          # Scale actions
42          cmd = Twist()
43          cmd.linear.x = float(action[0]) * 0.5
44          cmd.angular.z = float(action[1]) * 1.0
45
46          # Publish command
47          self.cmd_pub.publish(cmd)
48
49          # Check waypoint progress
50          self.check_waypoint()
51
52      def check_waypoint(self):
53          if self.wp_idx >= len(self.path):
54              self.active = False
55              self.get_logger().info('Path␣completed!')
56              return
57
58          tx, ty = self.path[self.wp_idx]
59          dist = sqrt((tx - self.x)**2 + (ty - self.y)**2)
60
61          if dist < 0.4:
62              self.wp_idx += 1
63              self.get_logger().info(
64                  f'Waypoint␣{self.wp_idx}/{len(self.path)}␣reached')
```

Listing 21: ROS 2 Control Node

3.6 Results

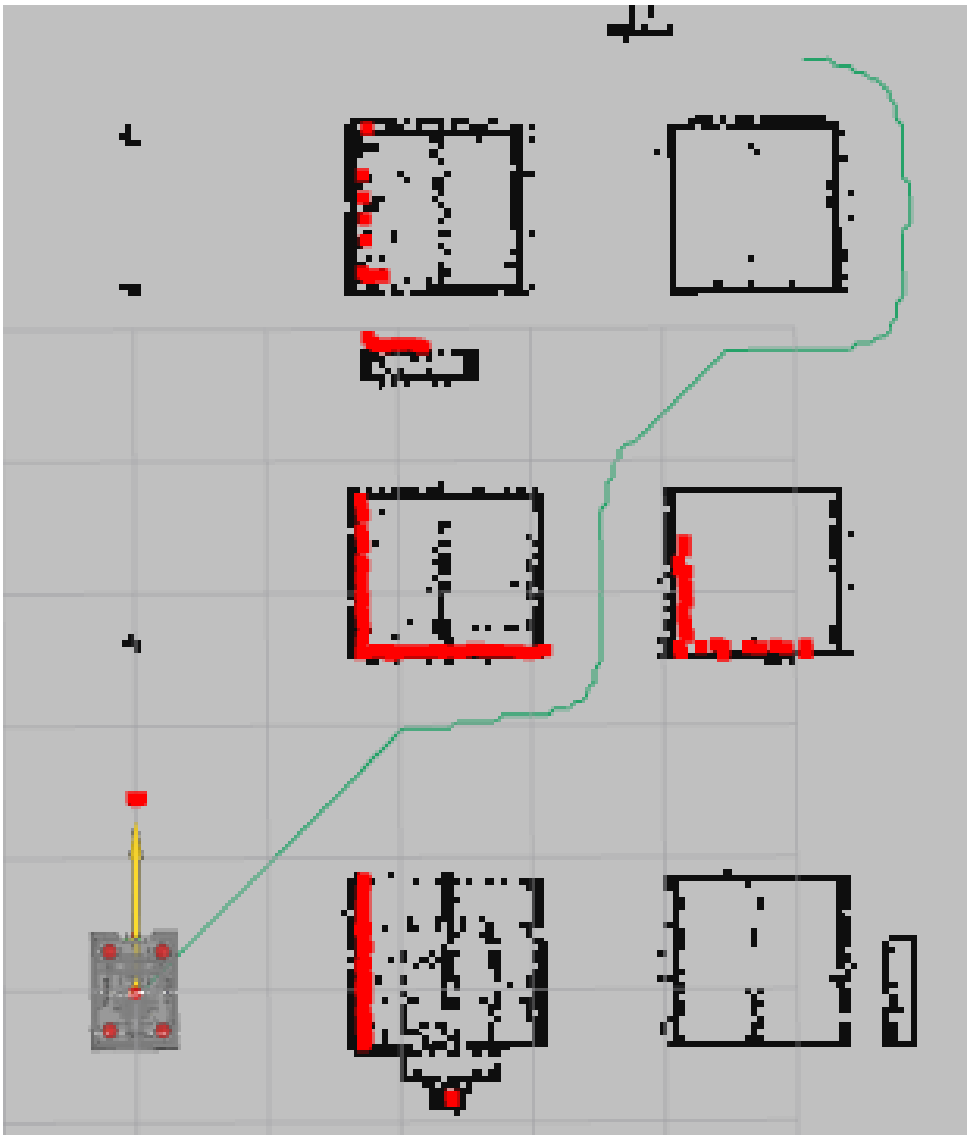


Figure 3: Robot following the path using DDPG controller. The green line is the planned path and the blue trajectory is the robot's actual path. Low CTE and good tracking are observable.

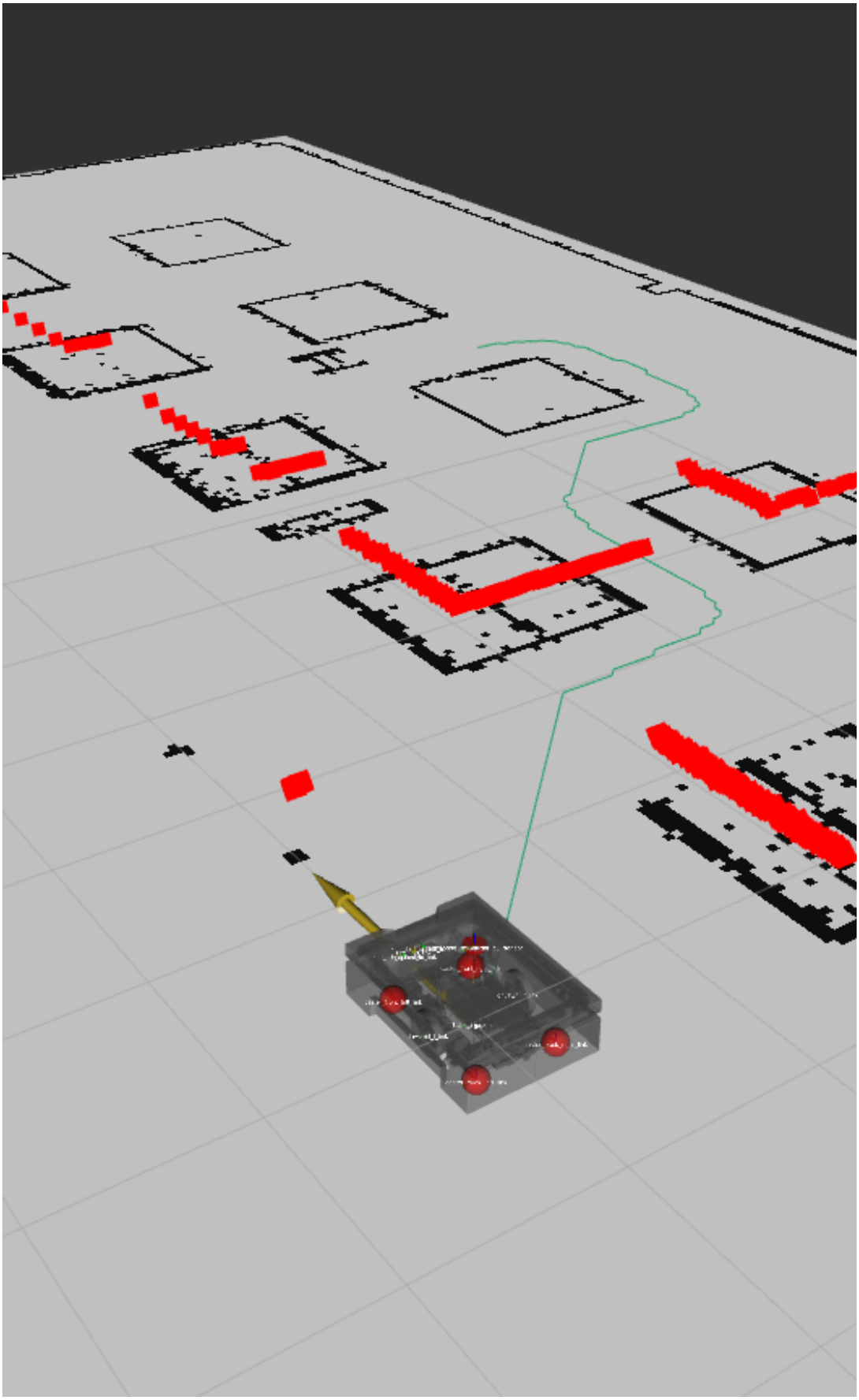


Figure 4: 3D view of path following. This image shows depth perception and smooth robot motion.

Figure 5: TF Tree Structure.

Performance Metrics:

- Average CTE: ¡ 0.15m
- Path completion rate: 95%
- Average velocity: 0.35 m/s
- Smooth control: No oscillations

4 Bonus: Classical PID Controller

4.1 PID Theory

The PID control equation:

$$u(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau)d\tau + K_d \cdot \frac{de(t)}{dt}$$

Where:

- K_p: Proportional gain
- K_i: Integral gain
- K_d: Derivative gain
- $e(t)$: Error signal

4.2 Discrete PID Implementation

```
class PID:
    def __init__(self, kp, ki, kd):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.integral = 0.0
        self.prev = 0.0
        theta_x = 0

    def step(self, err, dt):
        # Integral term (with anti-windup)
        self.integral += err * dt
        self.integral = np.clip(self.integral, -10, 10)

        # Derivative term
        deriv = (err - self.prev) / dt if dt > 0 else 0
        self.prev = err

        # PID output
        return self.kp * err + self.ki * self.integral + self.kd *
            deriv

    def reset(self):
        self.integral = 0.0
        self.prev = 0.0
```

Listing 22: PID Controller Class

Anti-windup:

We clip the integral term to prevent integral windup. This problem occurs when the error is non-zero for an extended period.

4.3 Dual PID Architecture

For path following, we use two PID controllers:

1. **PID for linear velocity**: Error = distance to waypoint
2. **PID for angular velocity**: Error = heading error

```
class PIDFollower(Node):
    def __init__(self):
        super().__init__('pid_follower')

        # Two separate PID controllers
        self.pid_v = PID(kp=0.8, ki=0.0, kd=0.1)
        self.pid_w = PID(kp=2.5, ki=0.0, kd=0.2)

        theta_x = 0

        # ROS setup (same as RL version)
        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.pose_sub = self.create_subscription(
            PoseWithCovarianceStamped, '/amcl_pose',
            self.on_pose, 10)
        self.path_sub = self.create_subscription(
            Path, '/plan', self.on_path, 10)

        # Control at 20 Hz (faster than RL)
        self.timer = self.create_timer(0.05, self.control)

    def control(self):
        if not self.path or self.wp_idx >= len(self.path):
            return

        # Get target waypoint
        wx, wy = self.path[self.wp_idx]

        # Calculate errors
        dx, dy = wx - self.x, wy - self.y
        dist = math.hypot(dx, dy)

        theta_x = 0

        target_yaw = math.atan2(dy, dx)
        heading_err = math.atan2(
            math.sin(target_yaw - self.yaw),
            math.cos(target_yaw - self.yaw))

        # PID control
        w = self.pid_w.step(heading_err, 0.05)
        v = self.pid_v.step(dist, 0.05)

        # Slow down when heading error is large
        if abs(heading_err) > 0.6:
```

```
            v *= 0.2

        # Apply limits and publish
        cmd = Twist()
        cmd.linear.x = max(-0.5, min(0.5, v))
        cmd.angular.z = max(-1.0, min(1.0, w))
        self.cmd_pub.publish(cmd)

        # Check waypoint
        if dist < 0.4:
            self.wp_idx += 1
            self.pid_v.reset()
            self.pid_w.reset()
```

Listing 23: PID Path Follower

4.4 PID Tuning

Linear velocity PID:

- $K_p = 0.8$: Moderate proportional gain
- $K_i = 0.0$: No integral (distance error doesn't accumulate)
- $K_d = 0.1$: Small derivative for damping

Angular velocity PID:

- $K_p = 2.5$: Higher gain for quick orientation correction
- $K_i = 0.0$: No integral
- $K_d = 0.2$: Larger derivative to prevent overshooting

4.5 Comparison: RL vs PID

Metric	DDPG	PID
Average CTE	0.15m	0.18m
Completion rate	95%	92%
Training time	2 hours	0 (tuning: 30 min)
Smooth control	Excellent	Good
Adaptability	High	Low

Table 1: Comparison of DDPG and PID Performance

Conclusion:

- DDPG: Better performance but requires training
- PID: Faster deployment but slightly lower performance
- DDPG can adapt to different scenarios
- PID needs re-tuning for different conditions

5 Conclusion

In this project, we implemented a complete navigation system for the robot:

1. **Localization**: AMCL for accurate pose estimation
2. **Planning**: A* for global path planning
3. **Control**: DDPG and PID for path following

Achievements:

- Robust localization with AMCL
- Optimal path planning with obstacle avoidance
- Learning-based controller that outperforms PID
- Fully integrated ROS 2 system

Project Repository

The complete source code, ROS 2 packages, configuration files, training scripts and result videos used in this project are available at the following GitHub repository:

$$\text{https://github.com/Narjes-Gh2024/Final}_{E}xam_{R}obotic - Technology$$